

Control Statements: Part 2; Logical Operators

Chapter 5 of C++ How to Program, 10/e

5.1 Introduction

- ▶ This chapter continues our presentation of structured-programming theory and principles.
- ▶ Demonstrates C++'s `for`, `do...while` and `switch` statements.
- ▶ We explore the essentials of counter-controlled iteration.
- ▶ We introduce the `break` and `continue` program-control statements.
- ▶ We discuss C++'s logical operators, which enable you to combine simple conditions in control statements.

5.2 Essentials of Counter-Controlled Iteration

- ▶ Counter-controlled iteration requires
 - a **control variable** (or loop counter)
 - the control variable's **initial value**
 - the control variable's **increment** that's applied during each iteration of the loop
 - the **loop-continuation condition** that determines if looping should continue.

```
1 // Fig. 5.1: WhileCounter.cpp
2 // Counter-controlled iteration with the while iteration statement.
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     unsigned int counter{1}; // declare and initialize control variable
8
9     while (counter <= 10) { // loop-continuation condition
10         cout << counter << " ";
11         ++counter; // increment control variable
12     }
13
14     cout << endl;
15 }
```

1 2 3 4 5 6 7 8 9 10

Fig. 5.1 | Counter-controlled iteration with the `while` iteration statement.

5.3 for Iteration Statement

- ▶ The **for iteration statement** (Fig. 5.2) specifies the counter-controlled iteration details in a single line of code.
- ▶ The initialization occurs once when the loop is encountered.
- ▶ The condition is tested next and each time the body completes.
- ▶ The body executes if the condition is true.
- ▶ The increment occurs after the body executes.
- ▶ Then, the condition is tested again.

```
1 // Fig. 5.2: ForCounter.cpp
2 // Counter-controlled iteration with the for iteration statement.
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     // for statement header includes initialization,
8     // loop-continuation condition and increment
9     for (unsigned int counter{1}; counter <= 10; ++counter) {
10         cout << counter << " ";
11     }
12
13     cout << endl;
14 }
```

1 2 3 4 5 6 7 8 9 10

Fig. 5.2 | Counter-controlled iteration with the for iteration statement.

for Statement Header Components

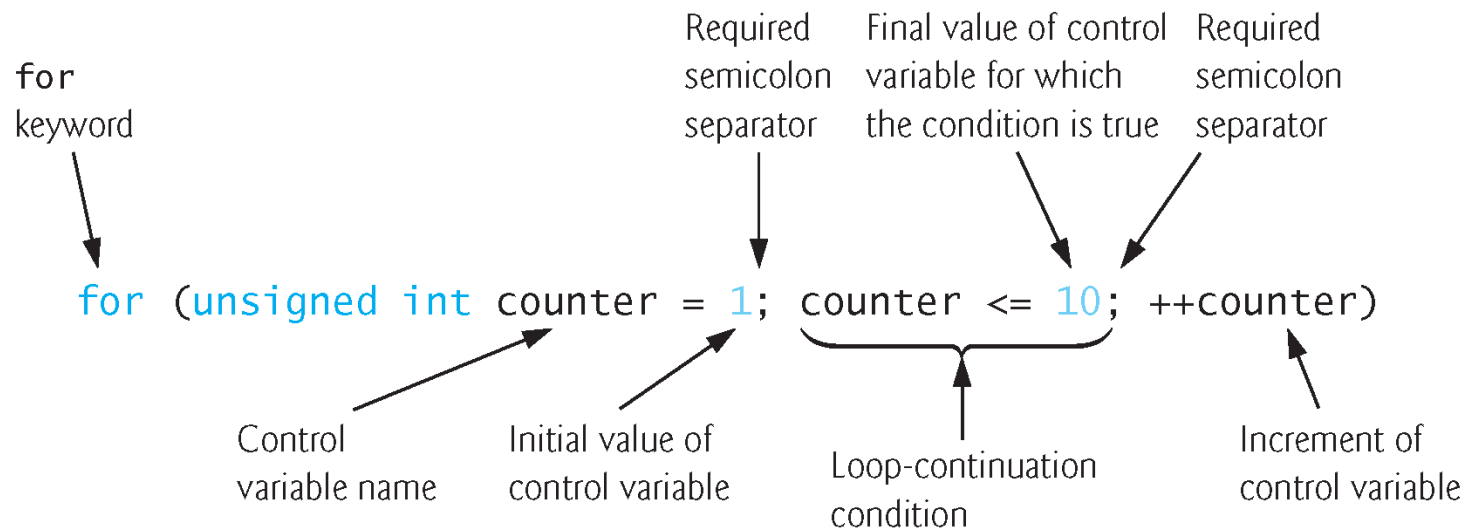


Fig. 5.3 | `for` statement header components.

5.3 for Iteration Statement (cont.)

- ▶ If the *initialization* expression declares the control variable, the control variable can be used *only* in the body of the **for** statement—the control variable will be unknown *outside* the **for** statement.
- ▶ This restricted use of the control variable name is known as the variable's **scope**.
- ▶ The scope of a variable specifies *where* it can be used in a program.

5.3 for Iteration Statement (cont.)

- ▶ The three expressions in the `for` statement header are optional (but the two semicolon separators are *required*).
- ▶ If the *loopContinuationCondition* is omitted, C++ assumes that the condition is true, thus creating an *infinite loop*.
- ▶ One might omit the *initialization* expression if the control variable is initialized earlier in the program.
- ▶ One might omit the *increment* expression if the increment is calculated by statements in the body of the `for` or if no increment is needed.



Common Programming Error 5.3

Placing a semicolon immediately to the right of the right parenthesis of a `for` header makes that `for`'s body an empty statement. This is normally a logic error.

5.3 for Iteration Statement (cont.)

- ▶ The “increment” of a **for** statement can be negative, in which case it’s really a *decrement* and the loop actually counts *downward*.
- ▶ If the loop-continuation condition is *initially false*, the body of the **for** statement is not performed.

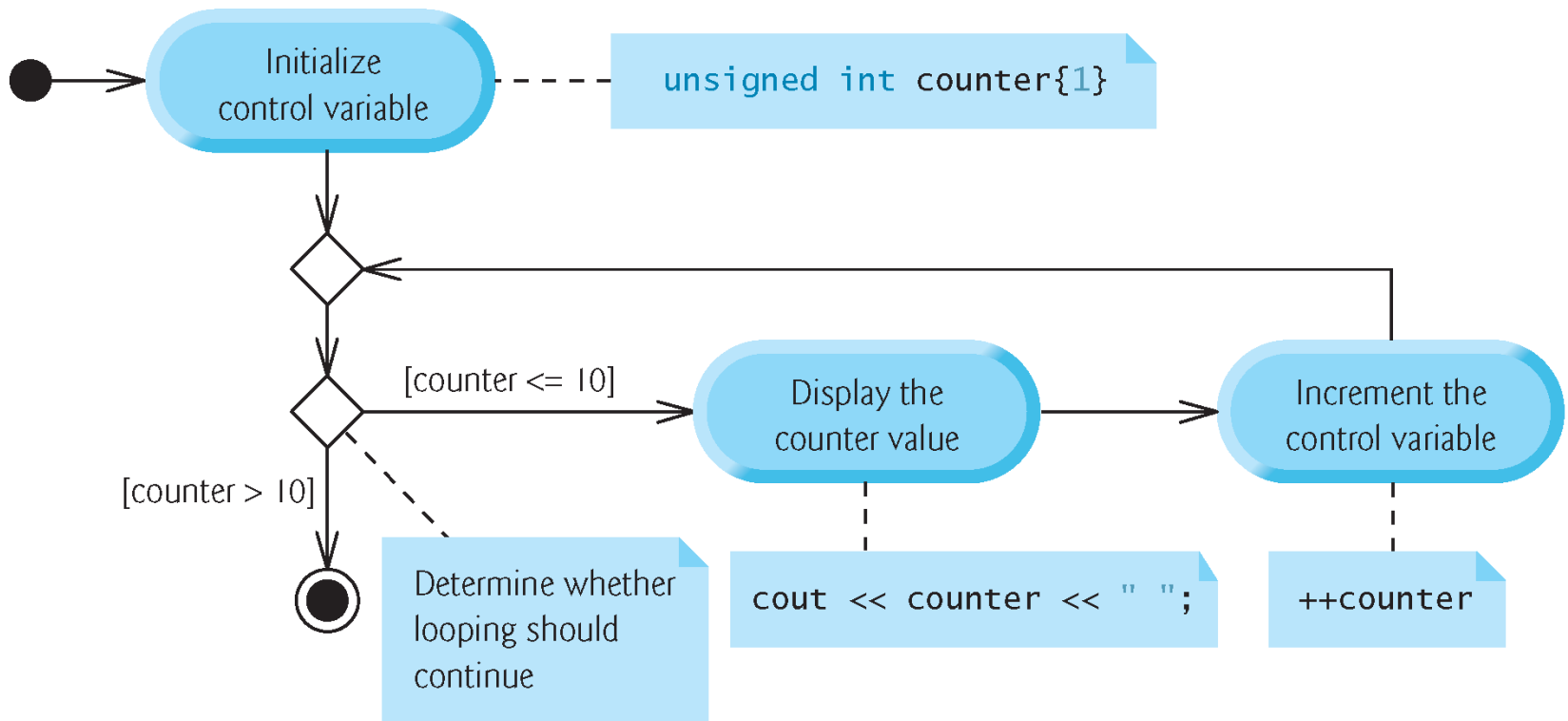


Fig. 5.4 | UML activity diagram for the for statement in Fig. 5.2.

5.4 Examples Using the for Statement

- ▶ Count from 1 to 100 in increments of 1.
 - `for (unsigned int i = 1; i <= 100; ++i)`
- ▶ Count from 100 down to 1 in decrements of 1.
 - `for (unsigned int i = 100; i >= 1; --i)`
- ▶ Count from 7 to 77 in steps of 7.
 - `for (unsigned int i = 7; i <= 77; i += 7)`
- ▶ Count from 20 down to 2 in steps of -2.
 - `for (unsigned int i = 20; i >= 2; i -= 2)`
- ▶ Iterate over the sequence 2, 5, 8, 11, 14, 17, 20.
 - `for (unsigned int i = 2; i <= 20; i += 3)`
- ▶ Iterate over the sequence 99, 88, 77, 66, 55, 44, 33, 22, 11, 0. We use `int` rather than `unsigned int` here because the condition does not become false until `i`'s value is -11, so the control variable must be able to represent both positive and negative values.
 - `for (int i = 99; i >= 0; i -= 11)`

5.5 Application: Summing Integers

- ▶ Write a program that takes a number n from the user and finds the sum of all the numbers from 1 to n and prints it.
- ▶ Modify the program so that it also prints the product of the numbers. How high can n be?
- ▶ How can we modify our variables to be able to use higher n ?

5.6 Application: Compound-Interest Calculations

- ▶ Consider the following problem statement:
 - A person invests \$1000.00 in a savings account yielding 5% interest. Assuming that all interest is left on deposit in the account, calculate and print the amount of money in the account at the end of each year for 10 years. Use the following formula for determining these amounts:
$$a = p (1 + r)^n$$
where
 - p* is the original amount invested (i.e., the principal),
 - r* is the annual interest rate,
 - n* is the number of years and
 - a* is the amount on deposit at the end of the *n*th year.
 - This problem involves a loop that performs the indicated calculation for each of the 10 years the money remains on deposit (Fig. 5.6).

5.6 Application: Compound-Interest Calculations (cont.)

- ▶ C++ does *not* include an exponentiation operator, so we use the **standard library function `pow`**.
 - `pow(x, y)` calculates the value of `x` raised to the `yth` power.
 - Takes two arguments of type `double` and returns a `double` value.
- ▶ This program will not compile without including header file `<cmath>`.
 - Includes information that tells the compiler to convert the value of `year` to a temporary `double` representation before calling the function.
 - Contained in `pow`'s function prototype.

5.6 Application: Compound-Interest Calculations (cont.)

Using Stream Manipulators to Format Numeric Output

- ▶ Parameterized stream manipulators `setprecision` and `setw` and the nonparameterized stream manipulator `fixed`.
- ▶ The stream manipulator `setw(4)` specifies that the next value output should appear in a **field width** of 4—i.e., `cout` prints the value with at least 4 character positions.
 - If less than 4 character positions wide, the value is **right justified** in the field by default.
 - If more than 4 character positions wide, the field width is extended *rightward* to accommodate the entire value.
- ▶ To indicate that values should be output **left justified**, simply output nonparameterized stream manipulator `left`.
- ▶ Right justification can be restored by outputting nonparameterized stream manipulator `right`.

5.6 Application: Compound-Interest Calculations (cont.)

- ▶ Stream manipulator `fixed` indicates that floating-point values should be output as fixed-point values with decimal points.
- ▶ Stream manipulator `setprecision` specifies the number of digits to the right of the decimal point.
- ▶ Stream manipulators `fixed` and `setprecision` remain in effect until they're changed—such settings are called **sticky settings**.
- ▶ The field width specified with `setw` applies only to the next value output.

5.6 Application: Compound-Interest Calculations (cont.)

- ▶ Variables of type `float` represent single-precision floating-point numbers and have approximately seven significant digits on most of today's systems.
- ▶ Variables of type `double` represent double-precision floating-point numbers.
 - These require twice as much memory as `float` variables and provide approximately 15 significant digits on most of today's systems—approximately double the precision of `float` variables.

5.6 Application: Compound-Interest Calculations (cont.)

- ▶ Most programmers represent floating-point numbers with type `double`.
- ▶ C++ treats all floating-point numbers you type in a program's source code (such as 7.33 and 0.0975) as `double` values by default.
- ▶ Such values in the source code are known as floating-point literals.

5.6 Application: Compound-Interest Calculations (cont.)

- ▶ In conventional arithmetic, floating-point numbers often arise as a result of division—when we divide 10 by 3, the result is $3.3333333\dots$, with the sequence of 3s repeating infinitely.
- ▶ The computer allocates only a fixed amount of space to hold such a value, so clearly the stored floating-point value can be only an approximation.
- ▶ `double` suffers from what we call representational error.



Common Programming Error 5.6

Using floating-point numbers in a manner that assumes they're represented exactly (e.g., using them in comparisons for equality) can lead to incorrect results. Floating-point numbers are represented only approximately.

5.6 Application: Compound-Interest Calculations (cont.)

- ▶ Simple explanation of what can go wrong when using double (or float) to represent dollar amounts (assuming that dollar amounts are displayed with two digits to the right of the decimal point):
 - Two calculated double dollar amounts stored in the machine could be 14.234 (which would normally be rounded to 14.23 for display purposes) and 18.673 (which would normally be rounded to 18.67 for display purposes).
 - When these amounts are added, they produce the internal sum 32.907, which would normally be rounded to 32.91 for display purposes. Thus, your output could appear as
$$\begin{array}{r} 14.23 \\ + 18.67 \\ \hline 32.91 \end{array}$$
 - A person adding the individual numbers as displayed would expect the sum to be 32.90.



Error-Prevention Tip 5.6

Do not use variables of type `double` (or `float`) to perform precise monetary calculations. The imprecision of floating-point numbers can lead to errors.

5.8 do...while Iteration Statement

- ▶ Similar to the `while` statement.
- ▶ The `do...while` statement tests the loop-continuation condition *after* the loop body executes; therefore, *the loop body always executes at least once*.
- ▶ Figure 5.9 uses a `do...while` statement to print the numbers 1–10.

```
1  // Fig. 5.9: DoWhileTest.cpp
2  // do...while iteration statement.
3  #include <iostream>
4  using namespace std;
5
6  int main() {
7      unsigned int counter{1};
8
9      do {
10         cout << counter << " ";
11         ++counter;
12     } while (counter <= 10); // end do...while
13
14     cout << endl;
15 }
```

1 2 3 4 5 6 7 8 9 10

Fig. 5.9 | do...while iteration statement.

5.8 do...while Iteration Statement (cont.)

do...while Statement UML Activity Diagram

- ▶ Figure 5.10 contains the do...while statement's UML activity diagram, which makes it clear that the loop-continuation condition is not evaluated until after the loop performs its body at least once.

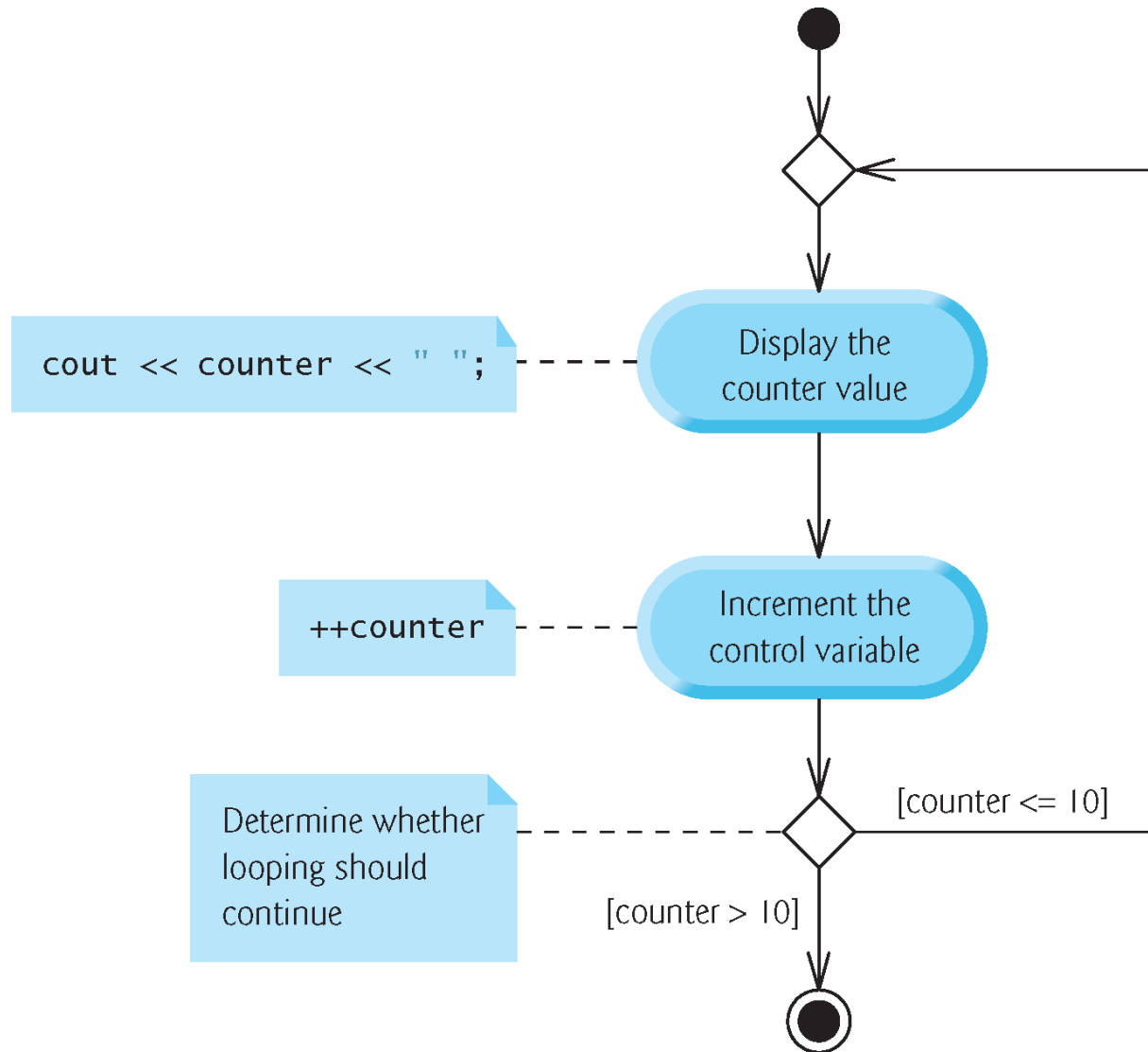


Fig. 5.10 | do...while iteration statement UML activity diagram.

5.9 switch Multiple-Selection Statement

- ▶ The **switch multiple-selection** statement performs many different actions based on the possible values of a variable or expression.
- ▶ Each action is associated with the value of an **integral constant expression** (i.e., any combination of character and integer constants that evaluates to a constant integer value).
- ▶ Figure 5.11 calculates the class average of a set of numeric grades entered by the user, and uses a switch statement to determine whether each grade is the equivalent of an A, B, C, D or F and to increment the appropriate grade counter.

```
1  // Fig. 5.11: LetterGrades.cpp
2  // Using a switch statement to count letter grades.
3  #include <iostream>
4  #include <iomanip>
5  using namespace std;
6
7  int main() {
8      int total{0}; // sum of grades
9      unsigned int gradeCounter{0}; // number of grades entered
10     unsigned int aCount{0}; // count of A grades
11     unsigned int bCount{0}; // count of B grades
12     unsigned int cCount{0}; // count of C grades
13     unsigned int dCount{0}; // count of D grades
14     unsigned int fCount{0}; // count of F grades
15
16     cout << "Enter the integer grades in the range 0-100.\n"
17           << "Type the end-of-file indicator to terminate input:\n"
18           << "    On UNIX/Linux/Mac OS X type <Ctrl> d then press Enter\n"
19           << "    On Windows type <Ctrl> z then press Enter\n";
20
21     int grade;
22
```

Fig. 5.11 | Using a switch statement to count letter grades. (Part 1 of 5.)

```
23 // loop until user enters the end-of-file indicator
24 while (cin >> grade) {
25     total += grade; // add grade to total
26     ++gradeCounter; // increment number of grades
27
28     // increment appropriate letter-grade counter
29     switch (grade / 10) {
30         case 9: // grade was between 90
31             case 10: // and 100, inclusive
32                 ++aCount;
33                 break; // exits switch
34
35         case 8: // grade was between 80 and 89
36             ++bCount;
37             break; // exits switch
38
39         case 7: // grade was between 70 and 79
40             ++cCount;
41             break; // exits switch
42
43         case 6: // grade was between 60 and 69
44             ++dCount;
45             break; // exits switch
46     }
```

Fig. 5.11 | Using a switch statement to count letter grades. (Part 2 of 5.)

```

47         default: // grade was less than 60
48             ++fCount;
49             break; // optional; exits switch anyway
50     } // end switch
51 } // end while
52
53 // set floating-point number format
54 cout << fixed << setprecision(2);
55
56 // display grade report
57 cout << "\nGrade Report:\n";
58
59 // if user entered at least one grade...
60 if (gradeCounter != 0) {
61     // calculate average of all grades entered
62     double average = static_cast<double>(total) / gradeCounter;
63
64     // output summary of results
65     cout << "Total of the " << gradeCounter << " grades entered is "
66         << total << "\nClass average is " << average
67         << "\nNumber of students who received each grade:"
68         << "\nA: " << aCount << "\nB: " << bCount << "\nC: " << cCount
69         << "\nD: " << dCount << "\nF: " << fCount << endl;
70 }

```

Fig. 5.11 | Using a switch statement to count letter grades. (Part 3 of 5.)


```
71     else { // no grades were entered, so output appropriate message
72         cout << "No grades were entered" << endl;
73     }
74 }
```

Enter the integer grades in the range 0-100.

Type the end-of-file indicator to terminate input:

On UNIX/Linux/Mac OS X type <Ctrl> d then press Enter

On Windows type <Ctrl> z then press Enter

99
92
45
57
63
71
76
85
90
100
^Z

Grade Report:

Total of the 10 grades entered is 778

Class average is 77.80

Fig. 5.11 | Using a switch statement to count letter grades. (Part 4 of 5.)

Number of students who received each grade:

A: 4

B: 1

C: 2

D: 1

F: 2

Fig. 5.11 | Using a switch statement to count letter grades. (Part 5 of 5.)

5.9 switch Multiple-Selection Statement (cont.)

- ▶ Write a program that gives the day of the week. That is, 1 would be Monday, 2 Tuesday, ... , 7 would be Sunday. It should ask for a number:

Input week number(1 – 7): 2 (for example)

and outputs the name of the day:

Tuesday

- ▶ Write a program that asks for the number of the month and outputs the number of days in that month:

Input month number: 3 (for example)

Output: Total number of days = 31

5.9 switch Multiple-Selection Statement (cont.)

- ▶ On UNIX/Linux/Mac OS X systems, end-of-file is entered by typing the following sequence on a line by itself
 - <Ctrl> d
 - This notation means to simultaneously press both the Ctrl key and the d key.
- ▶ On Windows systems, end-of-file can be entered by typing
 - <Ctrl> z
 - Windows typically displays the characters ^Z on the screen in response to this key combination

5.9 switch Multiple-Selection Statement (cont.)

- ▶ The `switch` statement consists of a series of `case labels` and an optional `default case`.
- ▶ When the flow of control reaches the `switch`, the program evaluates the `controlling expression` in the parentheses.
- ▶ Compares the value of the controlling expression with each `case` label.
- ▶ If a match occurs, the program executes the statements for that `case`.
- ▶ The `break` statement causes program control to proceed with the first statement after the `switch`.

5.9 `switch` Multiple-Selection Statement (cont.)

- ▶ Listing `cases` consecutively with no statements between them enables the `cases` to perform the same set of statements.
- ▶ Each `case` can have multiple statements.
 - The `switch` selection statement does not require braces around multiple statements in each `case`.
- ▶ Without `break` statements, each time a match occurs in the `switch`, the statements for that `case` *and* subsequent `cases` execute until a `break` statement or the end of the `switch` is encountered.
 - Referred to as “falling through” to the statements in subsequent `cases`.



Common Programming Error 5.7

Forgetting a break statement when one is needed in a switch is a logic error.

5.9 switch Multiple-Selection Statement (cont.)

- ▶ If *no* match occurs between the controlling expression's value and a `case` label, the `default` case executes.
- ▶ If no match occurs in a `switch` statement that does not contain a `default` case, program control continues with the first statement after the `switch`.



Error-Prevention Tip 5.7

In a switch, ensure that you test for all possible values of the controlling expression.

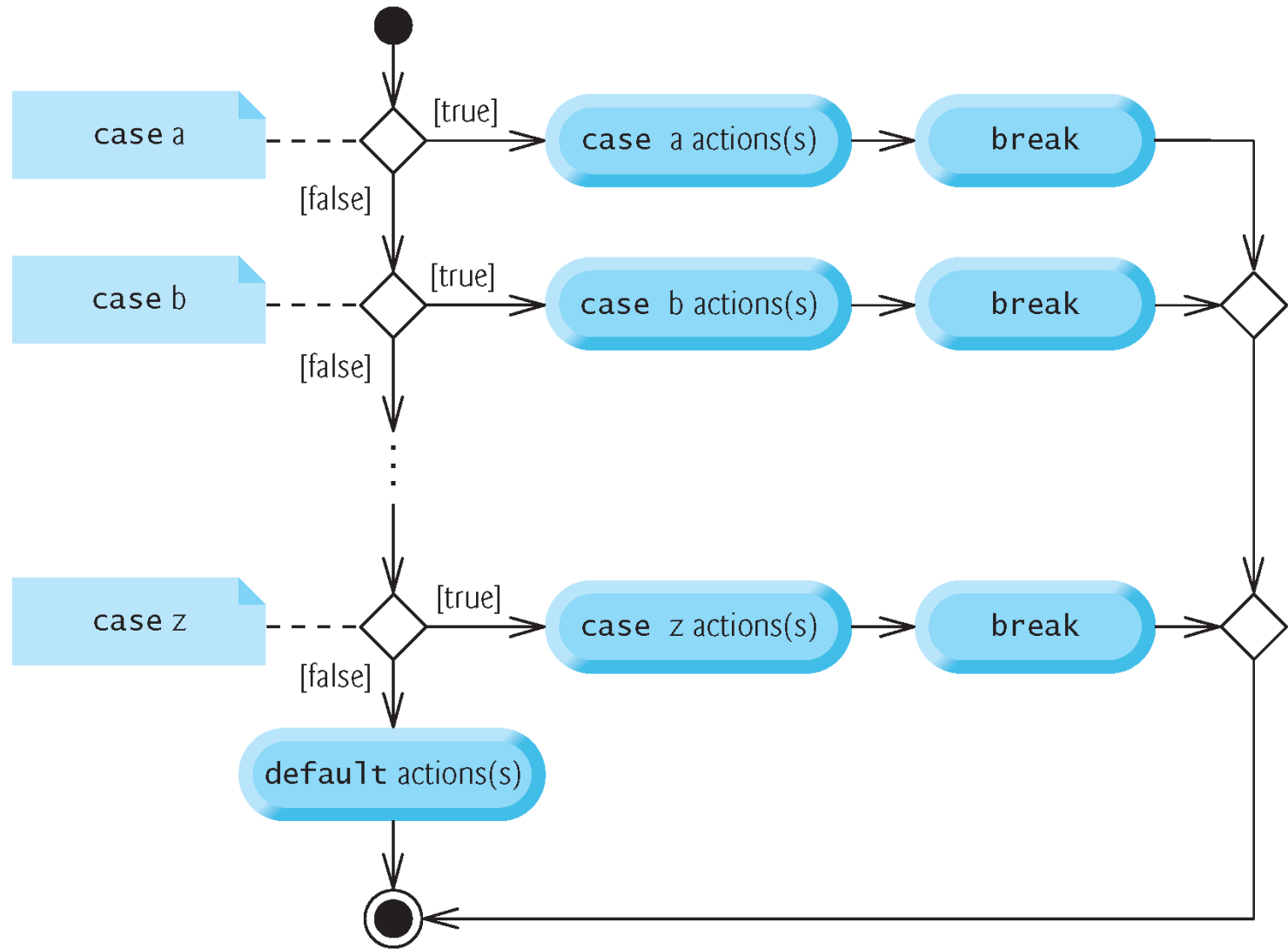


Fig. 5.12 | switch multiple-selection statement UML activity diagram with break statements.



Error-Prevention Tip 5.8

Provide a default case in switch statements. This focuses you on the need to process exceptional conditions.



Good Programming Practice 5.3

Although each case and the default case in a switch can occur in any order, place the default case last. When the default case is listed last, the break for that case is not required.

5.9 switch Multiple-Selection Statement (cont.)

► case values

- An integer constant is simply an integer value.
- In addition, you can use character constants—specific characters in single quotes, such as 'A', '7' or '\$'—which represent the integer values of characters and enum constants (introduced in Section 6.8). (Appendix B shows the integer values of the characters in the ASCII character set, which is a subset of the Unicode® character set.)
- The expression in each case also can be a **constant variable**—a variable containing a value which does not change for the entire program.
 - Such a variable is declared with keyword `const`

5.10 break and continue Statements

- ▶ In addition to selection and iteration statements, C++ provides statements `break` (which we discussed in the context of the `switch` statement) and `continue` to alter the flow of control.

5.10.1 break Statement

- ▶ The **break statement**, when executed in a `while`, `for`, `do...while` or `switch` statement, causes immediate exit from that statement.
- ▶ Program execution continues with the next statement.
- ▶ Common uses of the **break** statement are to escape early from a loop or to skip the remainder of a `switch` statement.
- ▶ Figure 5.13 demonstrates the **break** statement (line 13) exiting a `for` iteration statement.

```
1 // Fig. 5.13: BreakTest.cpp
2 // break statement exiting a for statement.
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     unsigned int count; // control variable also used after loop
8
9     for (count = 1; count <= 10; count++) { // loop 10 times
10         if (count == 5) {
11             break; // terminates loop if count is 5
12         }
13
14         cout << count << " ";
15     }
16
17     cout << "\nBroke out of loop at count = " << count << endl;
18 }
```

```
1 2 3 4
Broke out of loop at count = 5
```

Fig. 5.13 | break statement exiting a for statement.

5.10.2 continue Statement

- ▶ The **continue statement**, when executed in a **while**, **for** or **do...while** statement, skips the remaining statements in the body of that statement and proceeds with the next iteration of the loop.
- ▶ In **while** and **do...while** statements, the loop-continuation test evaluates immediately after the **continue** statement executes.
- ▶ In the **for** statement, the increment expression executes, then the loop-continuation test evaluates.

```

1  // Fig. 5.14: ContinueTest.cpp
2  // continue statement terminating an iteration of a for statement.
3  #include <iostream>
4  using namespace std;
5
6  int main() {
7      for (unsigned int count{1}; count <= 10; count++) { // loop 10 times
8          if (count == 5) {
9              continue; // skip remaining code in loop body if count is 5
10         }
11
12         cout << count << " ";
13     }
14
15     cout << "\nUsed continue to skip printing 5" << endl;
16 }

```

```

1 2 3 4 6 7 8 9 10
Used continue to skip printing 5

```

Fig. 5.14 | continue statement terminating an iteration of a for statement.

Continue and break exercises

- ▶ Write a program that prints all the numbers from 1 to 20 but uses a `while(true)` (that is, an infinite loop) and uses a `break` when it reaches 20.
- ▶ Modify the previous program using “`continue`” so that it skips 5 and 10.

5.11 Logical Operators

- ▶ C++ provides **logical operators** that are used to form more complex conditions by combining simple conditions.
- ▶ The logical operators are `&&` (logical AND), `||` (logical OR) and `!` (logical NOT, also called logical negation).

5.11.1 Logical AND (&&) Operator

- ▶ The **&&** (logical AND) operator is used to ensure that two conditions are *both* **true** before we choose a certain path of execution.
- ▶ The simple condition to the left of the **&&** operator evaluates first.
- ▶ If necessary, the simple condition to the right of the **&&** operator evaluates next.
- ▶ The right side of a logical AND expression is evaluated only if the left side is **true**.

5.11.2 Logical OR (| |) Operator

- ▶ The `||` (logical OR) operator determines if either *or* both of two conditions are `true` before we choose a certain path of execution.
- ▶ The `&&` operator has a higher precedence than the `||` operator.
- ▶ Both operators associate from left to right.

5.11.4 Logical Negation (!) Operator

- ▶ The ! (logical negation, also called logical NOT or logical complement) operator “reverses” the meaning of a condition.
- ▶ Unlike the logical operators && and | |, which are binary operators that combine two conditions, the logical negation operator is a unary operator that has only one condition as an operand.

5.11.5 Logical Operators Example

- ▶ Figure 5.18 demonstrates the logical operators by producing their truth tables.
- ▶ The output shows each expression that is evaluated and its `bool` result.
- ▶ By default, `bool` values `true` and `false` are displayed by `cout` and the stream insertion operator as `1` and `0`, respectively.
- ▶ **Stream manipulator `boolalpha`** (a sticky manipulator) specifies that the value of each `bool` expression should be displayed as either the word “true” or the word “false.”

```
1 // Fig. 5.18: LogicalOperators.cpp
2 // Logical operators.
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     // create truth table for && (logical AND) operator
8     cout << boolalpha << "Logical AND (&&)"
9         << "\nfalse && false: " << (false && false)
10        << "\nfalse && true: " << (false && true)
11        << "\ntrue && false: " << (true && false)
12        << "\ntrue && true: " << (true && true) << "\n\n";
13
14     // create truth table for || (logical OR) operator
15     cout << "Logical OR (||)"
16         << "\nfalse || false: " << (false || false)
17         << "\nfalse || true: " << (false || true)
18         << "\ntrue || false: " << (true || false)
19         << "\ntrue || true: " << (true || true) << "\n\n";
```

Fig. 5.18 | Logical operators. (Part I of 2.)

```
20
21 // create truth table for ! (logical negation) operator
22 cout << "Logical negation (!)"
23     << "\n!false: " << (!false)
24     << "\n!true: " << (!true) << endl;
25 }
```

```
Logical AND (&&)
false && false: false
false && true: false
true && false: false
true && true: true
```

```
Logical OR (||)
false || false: false
false || true: true
true || false: true
true || true: true
```

```
Logical negation (!)
!false: true
!true: false
```

Fig. 5.18 | Logical operators. (Part 2 of 2.)

5.11.5 Logical Operators (cont.)

- ▶ Figure 5.19 adds the logical and comma operators to the operator precedence and associativity chart.
- ▶ The operators are shown from top to bottom, in decreasing order of precedence.

Operators	Associativity	Type
:: ()	left to right <i>[See caution in Fig. 2.10 regarding grouping parentheses.]</i>	primary
++ -- <code>static_cast<type>()</code>	left to right	postfix
++ -- + - !	right to left	unary (prefix)
* / %	left to right	multiplicative
+ -	left to right	additive
<< >>	left to right	insertion/extraction
< <= > >=	left to right	relational
== !=	left to right	equality
&&	left to right	logical AND
	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment
,	left to right	comma

Fig. 5.19 | Operator precedence and associativity.

5.12 Confusing the Equality (==) and Assignment (=) Operators

- ▶ Accidentally swapping the operators == (equality) and = (assignment).
- ▶ Damaging because they ordinarily do not cause syntax errors.
- ▶ Rather, statements with these errors tend to compile correctly and the programs run to completion, often generating incorrect results through runtime logic errors.
- ▶ *Any nonzero value is interpreted as true.*

5.12 Confusing the Equality (==) and Assignment (=) Operators (cont.)

- ▶ Variable names are said to be *lvalues* (for “left values”) because they can be used on the *left* side of an assignment operator.
- ▶ Constants are said to be *rvalues* (for “right values”) because they can be used on only the *right* side of an assignment operator.
- ▶ *Lvalues* can also be used as *rvalues*, but not vice versa.



Error-Prevention Tip 5.9

Programmers normally write conditions such as `x == 7` with the variable name (an lvalue) on the left and the literal (an rvalue) on the right. Placing the literal on the left, as in `7 == x`, enables the compiler to issue an error if you accidentally replace the `==` operator with `=`. The compiler treats this as a compilation error, because you can't change a literal's value.



Common Programming Error 5.9

Using operator `==` for assignment and using operator `=` for equality are logic errors.

5.13 Structured Programming Summary

- ▶ We've learned that structured programming produces programs that are easier than unstructured programs to understand, test, debug, modify, and even prove correct in a mathematical sense.
- ▶ Figure 5.20 uses activity diagrams to summarize C++'s control statements.
- ▶ The initial and final states indicate the *single entry point* and the *single exit point* of each control statement.

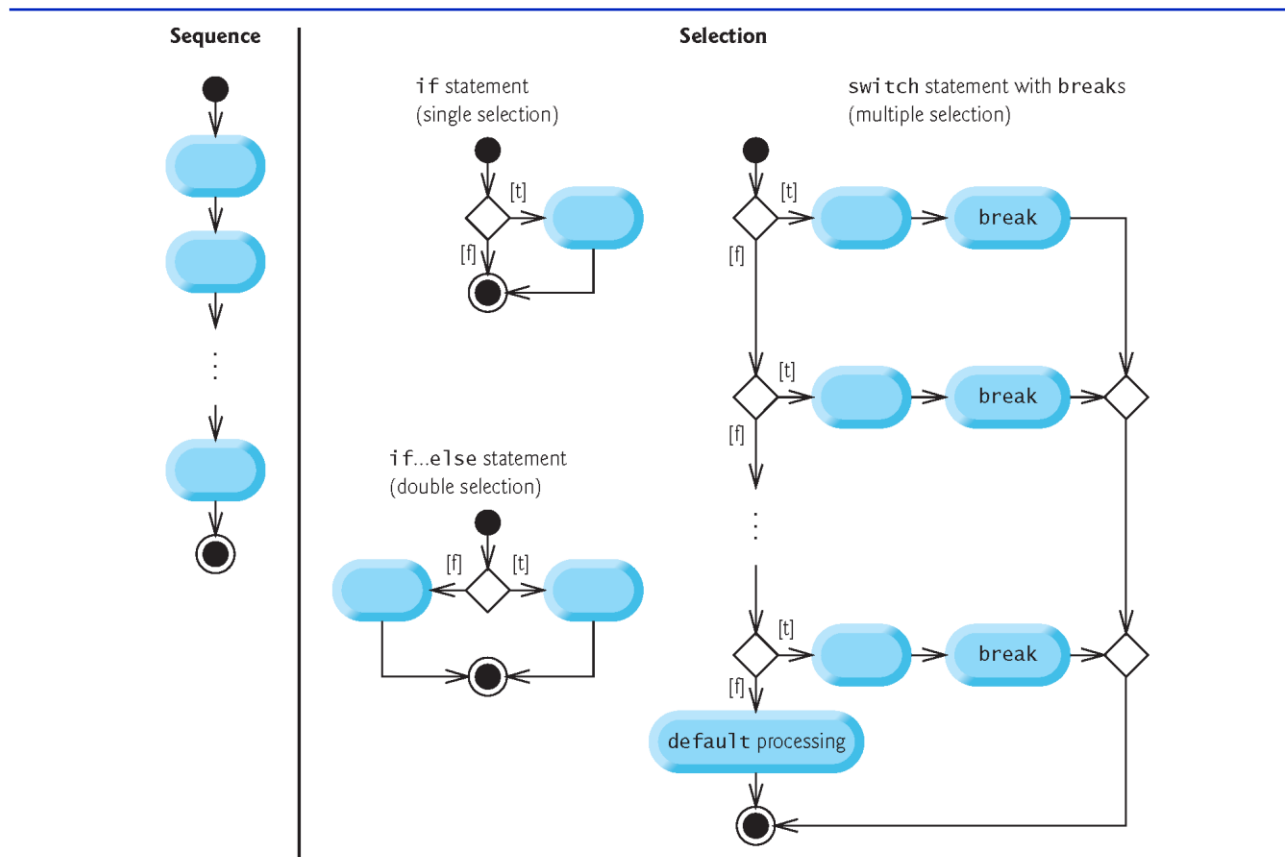
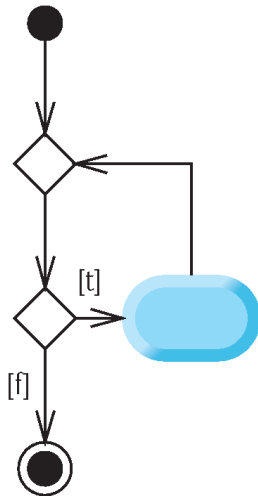


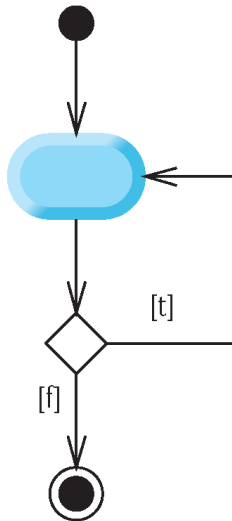
Fig. 5.20 | C++'s single-entry/single-exit sequence, selection and iteration statements. (Part 1 of 2.)

Repetition

while statement



do...while statement



for statement

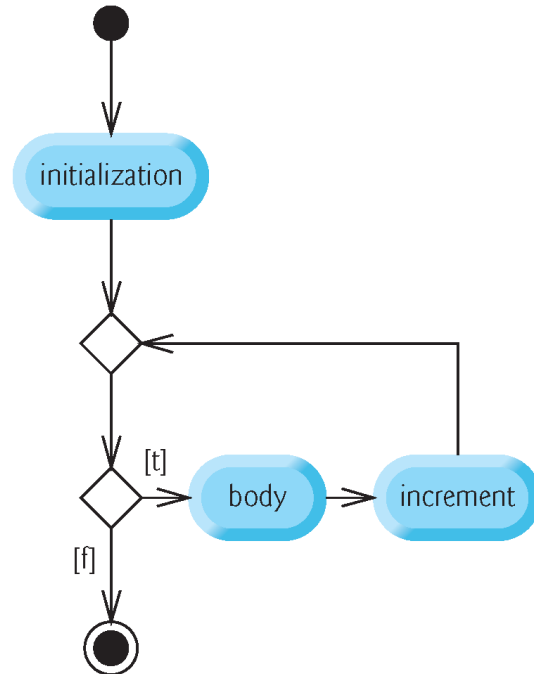


Fig. 5.20 | C++'s single-entry/single-exit sequence, selection and iteration statements. (Part 2 of 2.)

5.13 Structured Programming Summary (cont.)

- ▶ Figure 5.21 shows the rules for forming structured programs.
- ▶ The rules assume that action states may be used to indicate any action.
- ▶ The rules also assume that we begin with the so-called *simplest activity diagram* (Fig. 5.22), consisting of only an initial state, an action state, a final state and transition arrows.
- ▶ Applying the rules of Fig. 5.21 always results in an activity diagram with a neat, building-block appearance.
- ▶ Rule 2 generates a stack of control statements, so let's call Rule 2 the **stacking rule**.
- ▶ Rule 3 is the **nesting rule**.

Rules for forming structured programs

1. Begin with the simplest activity diagram (Fig. 5.22).
2. Any action state can be replaced by two action states in sequence.
3. Any action state can be replaced by any control statement (sequence of action states, if, if...else, switch, while, do...while or for).
4. Rules 2 and 3 can be applied as often as you like and in any order.

Fig. 5.21 | Rules for forming structured programs.

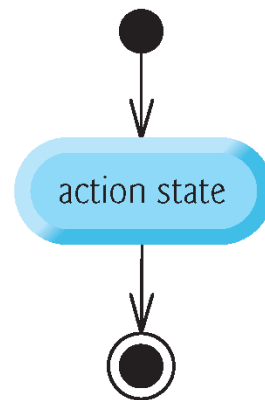


Fig. 5.22 | Simplest activity diagram.

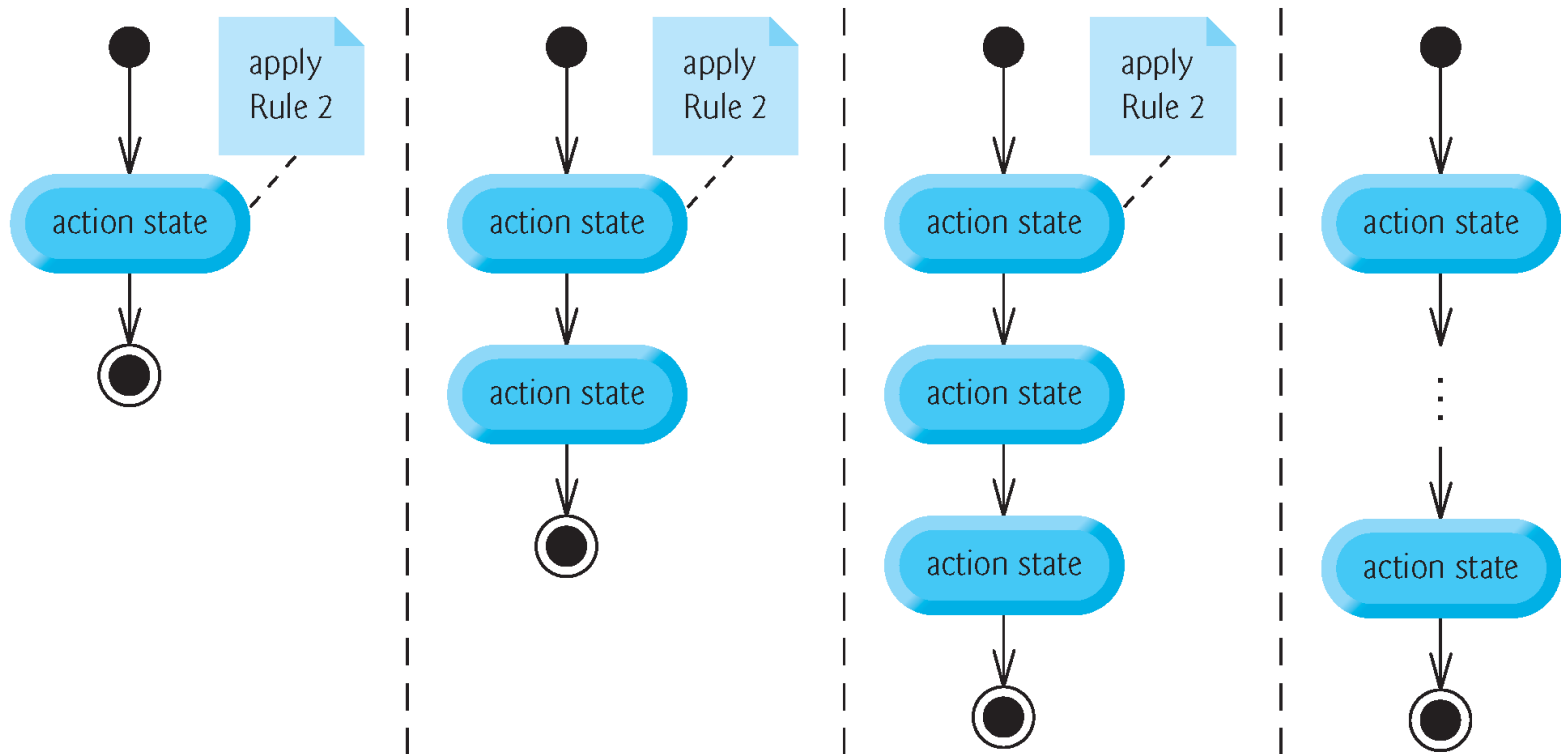


Fig. 5.23 | Repeatedly applying rule 2 of Fig. 5.21 to the simplest activity diagram.

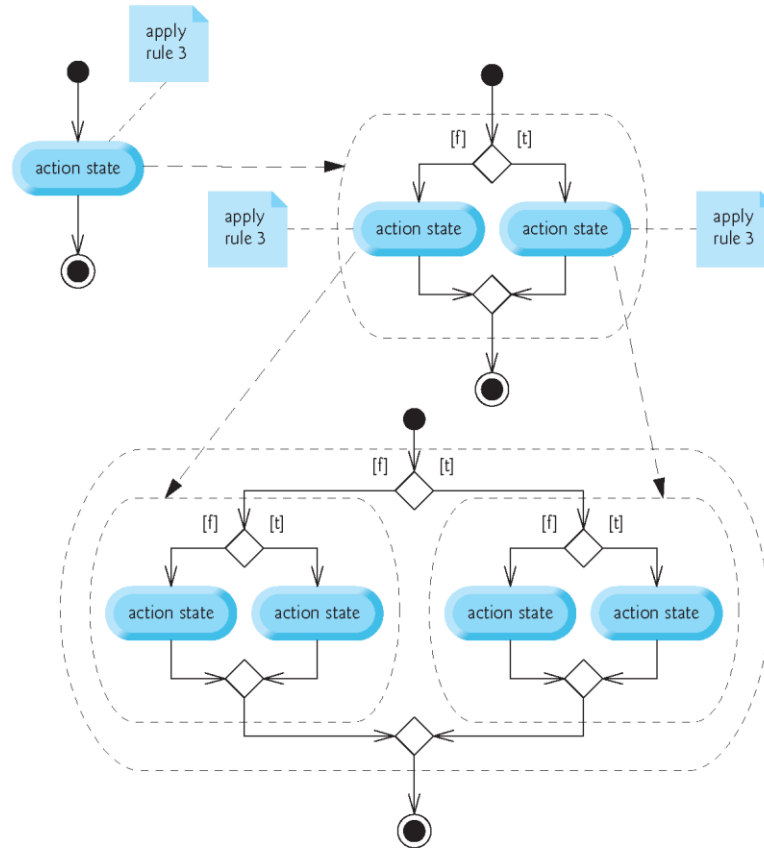


Fig. 5.24 | Repeatedly applying rule 3 of Fig. 5.21 to the simplest activity diagram.

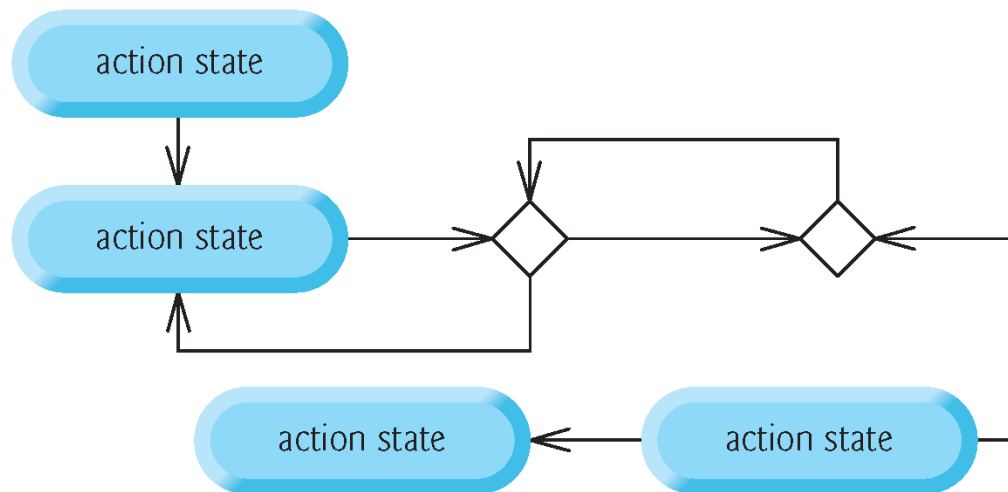


Fig. 5.25 | “Unstructured” activity diagram.